The Bixi Crew

Speaker notes — final presentation

BIXI Demand MLOps Platform

15-minute demand forecasting for every Montreal station,
departures & arrivals, productionised on AWS

Othmane Zizi	`othmane-zizi-pro`
Sarah Liu	`sarahliu-mma`
Ruihe "Louis" Zhang	`Mudkipython`
Rui Zhao	`ruizhaoca`

Repository	`github.com/ruizhaoca/bixi-demand-mlops-platform`
Live demo	`bixidemandlocal.streamlit.app`
Live demo (EC2)	`3.16.250.166:8501`

Target: 7–8 minutes, four presenters, ≈1.5–2 min each. Notes follow the deck slide order; the same notes are in the PPTX notes pane.

Sarah Liu — introduction & data *(≈2:00)*

Slide 0 — Title (shared open, Sarah leads)

- "We are **The Bixi Crew**. We forecast 15-minute BIXI demand for every Montreal station, separately for departures and arrivals, and turn it into a rebalancing tool — productionised end-to-end on AWS."
- Hand off the structure: Sarah (problem + data), Louis (AWS structure), Othmane (modelling), Rui (rebalancing + live app). Repo and two live demos are on screen.

Slide 1 — The rebalancing problem & why our design fits

- BIXI has ∼1,100+ stations; bikes pile up downtown and run out in residential areas, so riders hit empty or full docks.
- Operators rebalance by truck but need foresight at *operational* resolution. The course-1 prototype was hourly and only the top ∼400 stations — too coarse and partial.
- Four deliberate choices: **15-minute** (4× finer), **departures vs arrivals** separately (their gap is the signal), **all stations**, and **leakage-safe & productionised**.

Slide 2 — Data: sources, cleaning, and what demand looks like

- Sources: BIXI open-data trips (2024 + May/Oct 2025) joined to Open-Meteo 15-minute weather.
- Cleaned into a tidy 15-minute station-demand table, split into departures and arrivals.
- Heatmap = average demand by weekday × 15-minute slot: quiet overnight, building through midday, busiest on weekday evenings — exactly the structure our time features capture.

Slide 3 — Feature engineering, leakage-safe

- Cyclical slot-of-day / day-of-week / month; 2024 historical baselines (average + recent-lag) built **leave-one-out**; 15-minute weather; frequency + smoothed **target encoding** of the station name.
- Leakage-safe: strict temporal split — train 2024, validate May-2025, test Oct-2025 — with encoders and baselines fit on **train only** and applied forward. "Over to Louis."

Ruihe "Louis" Zhang — AWS & productionised structure *(≈1:45)*

Slide 4 — Architecture, all infrastructure-as-code

- Push to GitHub → Actions CI runs `pytest` and builds both Docker images.
- AWS in `us-east-2` is provisioned entirely by **AWS CDK** in four stacks: `BixiNetwork` (VPC), `BixiStorage` (S3 + SSM), `BixiMlflow` (MLflow on EC2 + S3), `BixiBatch` (ECR + AWS Batch).
- AWS Batch runs `python -m bixi.pipeline` over the full dataset from `s3://insy684`; serving is two Streamlit deployments (Community Cloud + EC2). No credentials in git — default boto3 chain (SSO locally, IAM role in cloud).

Slide 5 — Resumable, staged pipeline

- Eight ordered stages: ingest → features → data → train → explain → fairness → drift → register.
- Each stage writes a `_SUCCESS` marker to S3 — a run resumes from any step. Default run starts at `data` (cleaned data already in S3); full rebuild starts at `ingest`.
- Identical code local vs AWS Batch; CI smoke-tests the pipeline image with no AWS.

Slide 6 — Repository structure

- Single-responsibility modules in `src/bixi`, each owning one stage of the contract in `config.py`.
- Config-driven: env vars flip local subsample ↔ full cloud run with the same code. `infra/` is the CDK app; `docker/` pins the runtime; `tests/` runs on synthetic data. "Othmane takes the modelling."

Othmane Zizi — predictive modelling *(≈2:00)*

Slide 7 — Modelling: a multi-model search, auto-selected

- Not a hand-picked model: LightGBM + XGBoost baselines, **FLAML** AutoML, and **Optuna** Bayesian HPO, every trial logged to MLflow.
- Best by validation RMSE is selected and promoted automatically. Run once per target; both independently land on `lgbm_optuna` — a consistent, reproducible winner.

Slide 8 — MLflow tracking + registry

- Tracking server on EC2 with an S3 artifact store. Departure experiment = 73 runs, arrival = 57 — every candidate, FLAML and Optuna trial with params, metrics, model.
- Best run per target is registered and promoted to the **production** alias — the exact artifact the apps serve.

Slide 9 — Results

- Per split. 15-minute slot demand is genuinely noisy (many zero slots), so absolute R^2 is modest *by design* — but errors are well under **one trip per slot**.
- Beats a naive baseline on RMSE and R^2, stable across two unseen months (no overfitting), and strongest exactly where it matters — busy stations.

Slide 10 — Explainability: SHAP & LIME

- SHAP beeswarms: most signal from the 2024 historical baselines and cyclical time-of-day, with weather secondary — same sensible story for departures and arrivals.
- LIME force plots explain individual predictions on demand.

Slide 11 — Fairness & four-type drift

- Fairness: error parity (RMSE/MAE) across demand tiers and geography — accuracy concentrates in high-demand stations, low-tier R^2 near zero; surfaced so operators don't over-trust quiet-station forecasts.
- All four Evidently drift types — feature, target, prediction, concept — on Oct-2025 data. "Over to Rui."

Rui Zhao — rebalancing, live app & close *(≈2:00)*

Slide 12 — Net-flow rebalancing layer

- `net_flow = arrival − departure` per station per slot; cumulate across the day for a relative occupancy trajectory.
- Read peak **deficit** (needs bikes / stockout) and peak **surplus** (needs docks / overflow), rank by severity. Map: downtown core fills (needs docks); residential periphery drains (needs bikes).

Slide 13 — The Streamlit app (live demo)

- One shared UI, four pages: multi-day 15-minute forecast; rebalancing map + ranked list + per-station trajectory; custom what-if inputs; model monitoring (SHAP, fairness, drift).
- Runs on Community Cloud (committed artifacts, no AWS at runtime) and an EC2 container backed by S3. **Switch to the live app and click through the pages.**

Slide 14 — Meaning, limitations, conclusion

- Value: a daily ranked list of where to send a rebalancing truck first.
- Honest limitation: no dock capacity / real-time occupancy in the trip data, so the trajectory starts from a common zero — a **relative** ranking, not exact stockout times.
- MLOps maturity: tracking, registry, explainability, fairness, 4-type drift, CI, Docker, CDK — end to end. Next: live station-status feed for absolute occupancy; drift-triggered retrains.

Slide 15 — Thank you / Q&A (shared close)

- "Thank you — happy to take questions and walk through the live app." Repo and both live demos on screen; the team is The Bixi Crew: Othmane, Sarah, Louis, Rui.

Timing guide: keep each presenter to ≈1.5–2 minutes; the live demo (Slide 13) is the natural place to absorb slack or recover time. Total target 7–8 minutes.